EWB AWS
Project Report

Student Name	_Gagan K R_____
USN / Roll No.	_4GM24AI026_____
Class / Batch	_2024_____
Faculty Name	_EWB_____
AWS Region	Asia Pacific (Sydney) — ap-southeast-2 (confirm in console)
Submission Date	30/10/2026_____

Cross-Reference:Close Total Pages: 6 Blank Pages: 1 Pag

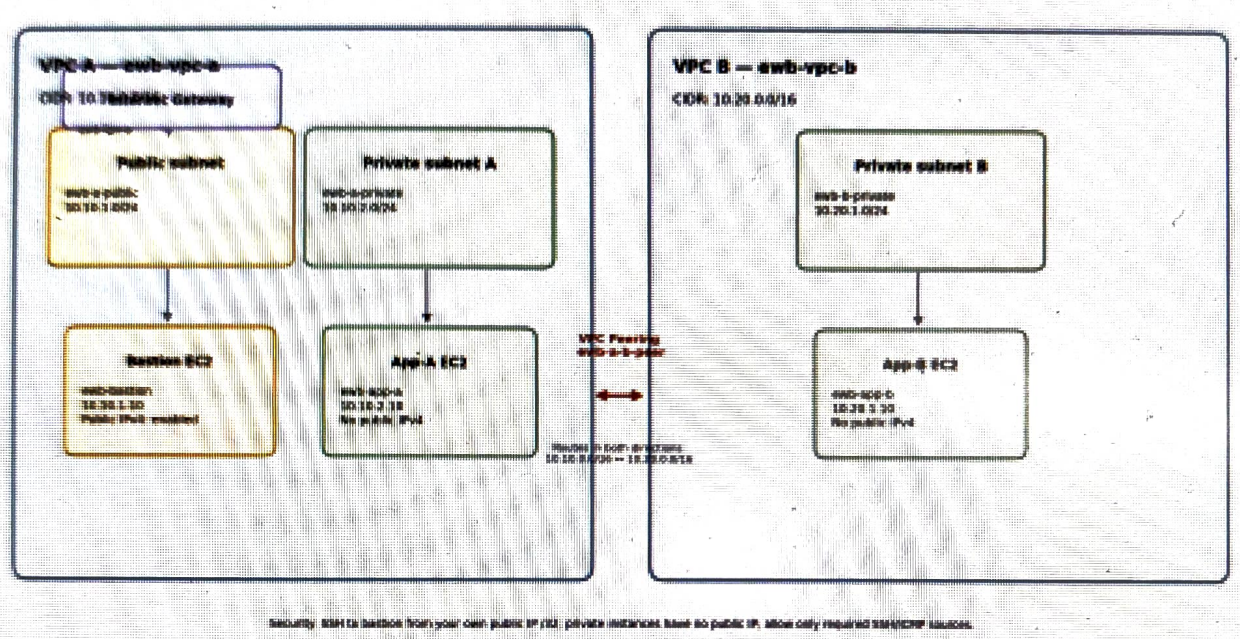
1. Aim

To create two Amazon Virtual Private Clouds (VPCs), connect them using VPC Peering, configure routes and security groups, and verify private communication between EC2 instances while keeping the application instances without public IPv4 addresses.

2. Architecture Diagram

The following diagram represents the intended topology from the project workbook. Confirm the actual resource names and addresses in your AWS console before final submission.

AWS VPC Peering Lab — Conceptual Architecture



Security: SSH to bastion from your own public IP /32; private instances have no public IP; allow only required traffic/ICMP sources.

3. Planned AWS Resource Inventory

VPC A	ewb-vpc-a	10.10.0.0/16
Public subnet A	ewb-a-public	10.10.1.0/24
Private subnet A	ewb-a-private	10.10.2.0/24
VPC B	ewb-vpc-b	10.20.0.0/16
Private subnet B	ewb-b-private	10.20.1.0/24
Internet Gateway	ewb-igw-a	Attached to VPC A
VPC peering	ewb-a-b-peer	VPC A ↔ VPC B; status should be Active
Bastion EC2	ewb-bastion	10.10.1.10; public IPv4 enabled
App-A EC2	ewb-app-a	10.10.2.10; no public IPv4
App-B EC2	ewb-app-b	10.20.1.10; no public IPv4
Public route table	ewb-rt-a-public	Local VPC A; 0.0.0.0/0 → IGW; 10.20.0.0/16 → peering
Private route table A	ewb-rt-a-private	Local VPC A; 10.20.0.0/16 → peering
Private route table B	ewb-rt-b-private	Local VPC B; 10.10.0.0/16 → peering

4. Routing and Security Design

- **Public route table:** retain the local route for 10.10.0.0/16, send 0.0.0.0/0 to the Internet Gateway, and send 10.20.0.0/16 to the peering connection.
- **Private route table A:** route 10.20.0.0/16 to the peering connection.
- **Private route table B:** route 10.10.0.0/16 to the peering connection.
- **Bastion security group:** inbound SSH TCP 22 only from the student's current public IPv4 address as /32 (AWS "My IP" option).
- **App-A and App-B security groups:** SSH TCP 22 only from the bastion private IP 10.10.1.10/32; ICMP Echo Request only from the peer application instance IP, as required by the lab.
- **Outbound rules:** use the workbook's default outbound allowance unless your instructor specifies otherwise. Do not expose SSH to 0.0.0.0/0.

5. Implementation Procedure

1. Create VPC A (10.10.0.0/16), its public and private subnets, and attach the Internet Gateway to VPC A.
2. Create the public route table and associate it with the public subnet; add the Internet Gateway default route.
3. Create VPC B (10.20.0.0/16), its private subnet, and its private route table.
4. Create the VPC peering connection between VPC A and VPC B, then accept it if prompted.
5. Add peering routes to the public route table and both private route tables.
6. Create the security groups with least-privilege SSH and ICMP rules.
7. Create or select the EC2 key pair. Store the private key securely on the laptop; never upload it to GitHub or include it in this report.
8. Launch the bastion in the public subnet and the two application instances in the private subnets. Use the instructor-approved instance type and check estimated costs before launching.
9. Verify that App-A and App-B have no public IPv4 addresses.
10. Run and record the baseline and post-peering connectivity tests. Capture genuine AWS console and terminal screenshots.

6. Verification Results

Peering status	Active	
Baseline ping before routes/peering	Should fail/time out; peering path is configured	
App-A → App-B ping after routes and ICMP rules	Replies received	
App-B → App-A ping after routes and ICMP rules	Replies received	
SSH to App-B through bastion	Successful, if configured as workbook requires	
Controlled ICMP rule removal	Ping fails while rule is removed	
Restore ICMP rule	Ping succeeds again	
Private instance public IPv4	None for App-A and App-B	

7. AWS Screenshot Evidence — Insert Actual Screenshots

Replace each placeholder with a screenshot captured from your own AWS console or terminal. Keep the AWS Region visible where practical, and do not show private keys, passwords, tokens, or other secrets.

Figure 1 — VPCs and CIDR blocks
EC2 VPC console showing both app-vpc-a and app-vpc-b with their CIDRs.

Figure 2 — Subnets
Subnets page showing all three subnets, their VPCs, and CIDR blocks.

Figure 3 — Route tables
Routes and subnet associations for all three route tables, including peering targets.

Figure 4 — VPC peering
Peering connection details with status Active and both VPC IDs.

Figure 5 — Security groups
Inbound rules for bastion, App-A, and App-B. Hide any unrelated sensitive information.

Figure 6 — EC2 instances
Instance list/details showing names, states, private IPs, and no public IPs for App-A/App-B.

Figure 7 — Connectivity test
Terminal output showing ping results in both directions after routes and rules are configured.

Figure 8 — SSH test
Terminal evidence showing SSH access to App-B through the bastion. Do not expose key contents.

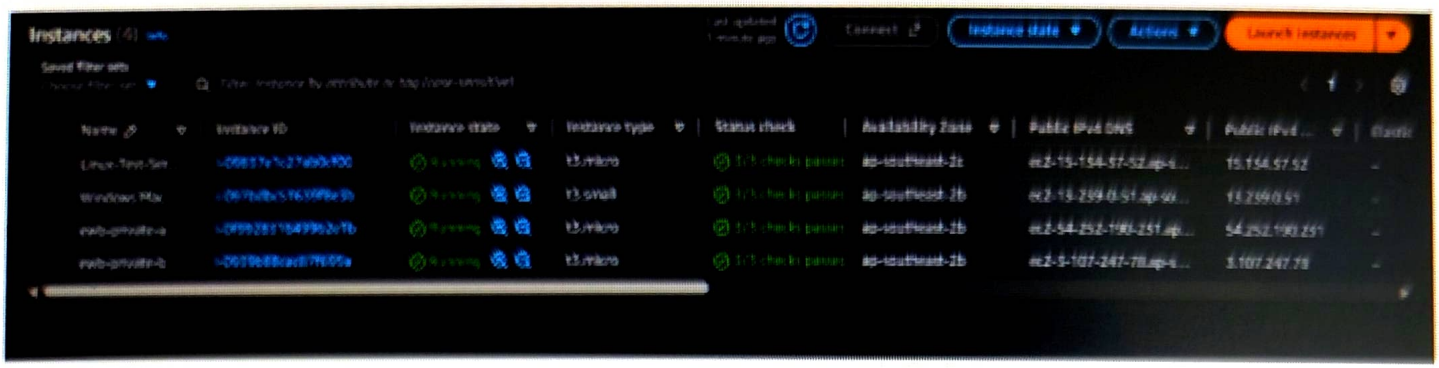
Figure 9 — Failure and recovery
Evidence of the temporary ICMP rule test and successful recovery after restoring the rule.

Figure 10 — Cleanup
Console evidence that billable test resources were stopped/terminated and unneeded resources cleaned up, if required.

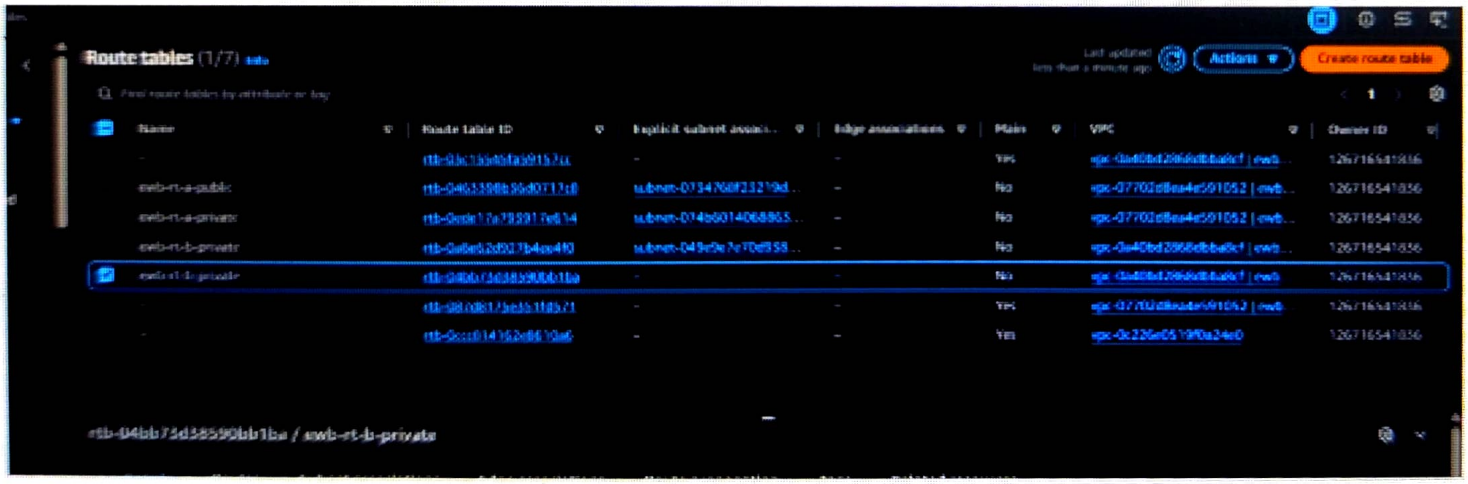
8. Results and Conclusion

The intended outcome is private communication between instances in two separate VPCs through VPC Peering, controlled by route tables and security groups. Complete this section only after testing: state which tests passed, which failed, how any errors were resolved, and whether the private instances remained without public IPv4 addresses. Do not claim success unless your actual test evidence supports it.

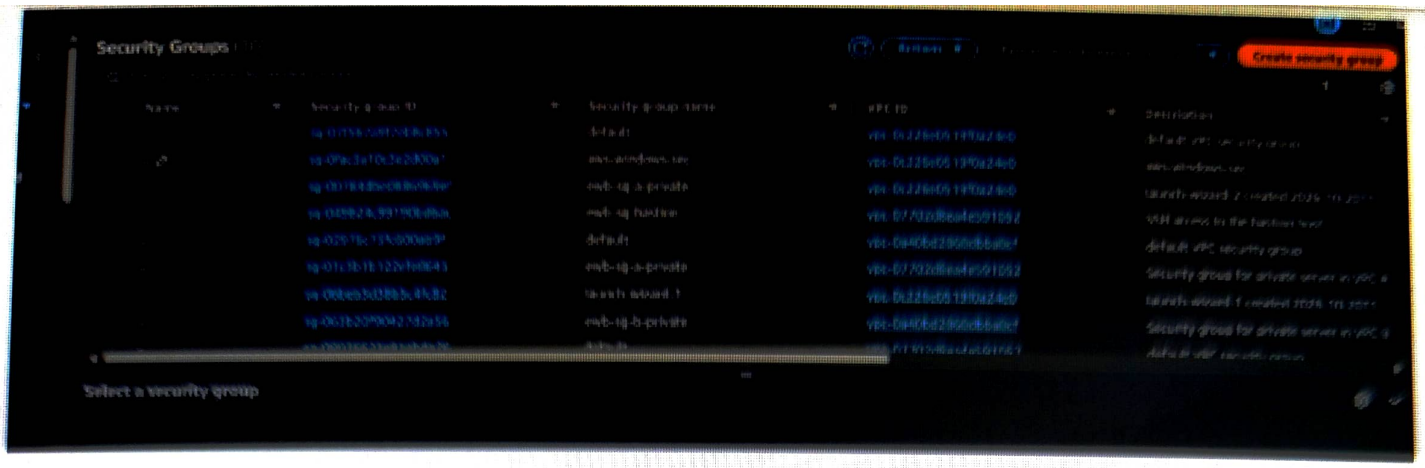
screenshots

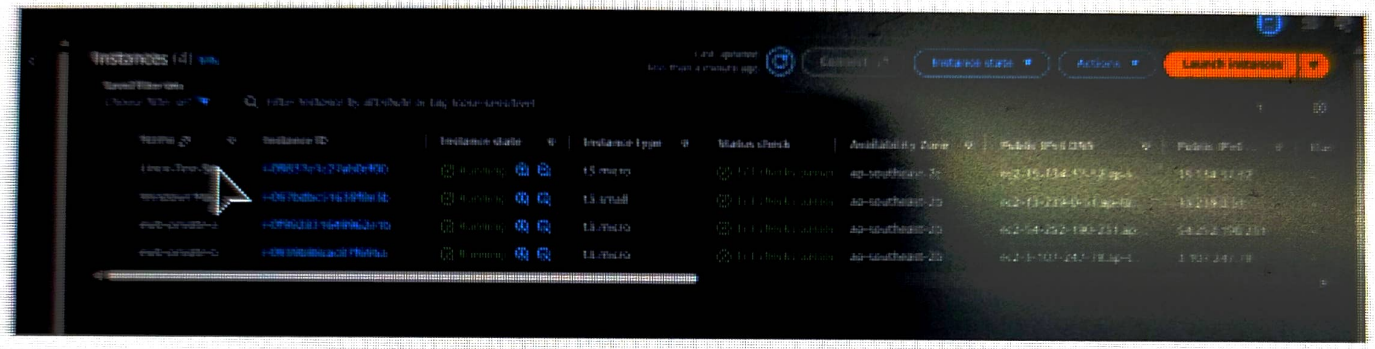









Security Groups
Select a security group


Instances